

Which is better when using in the Contract list? Do you use `override`?

When it comes to your `ContractList` class, the short answer is: **No, you do not use `@Override` for your main methods like `addContract()` or `addSalary()`.**

However, there is *one* place where using `@Override` makes your code look incredibly clean, smart, and professional without looking too advanced for Semester 2.

Let's break down exactly what `@Override` means, why you don't use it inside `ContractList`, and where you *should* use it to impress your teacher.

What does `@Override` actually mean?

In Java, `@Override` is a safety tag. It tells the compiler: *"A version of this method already exists in a parent class, but I want to replace it with my own version."*

Because your methods in `ContractList` (like `addContract` and `addSalary`) are brand-new methods that you invented yourself, Java has nothing to replace. If you try to put `@Override` above them, NetBeans will give you a red underline error.

Where SHOULD you use `@Override` ? (The `toString()` Trick)

Right now, in your `Contract` and `Salary` classes, you have custom display methods like `displayContractInfo()`.

If you want to make your code look much more like proper Semester 2 Object-Oriented Programming, you can replace those custom methods by overriding Java's built-in `toString()` method.

Every single class you create in Java automatically inherits from a hidden master class called the `Object` class. The `Object` class has a built-in method called `toString()`.

`Object` class. The `Object` class has a built-in method called `toString()` .

- **The Problem:** By default, if you try to print an object directly in your main method like `System.out.println(myContract);` , Java prints a jumbled memory address (like `Contract@1a2b3c`).
- **The Solution:** You use `@Override` to rewrite how Java prints your object!

How to use it in `Contract.java` :

Instead of a custom `displayContractInfo()` method, you write this:

Java



```
@Override
public String toString() {
    // Instead of printing directly, we pack all the text into one single String
    return "Contract ID: " + contractId + " | Player: " + playerId + " | Status: " +
}
```

Which is better for your `ContractList` ?

Let's look at how this changes how you print data inside your `ContractList` class, comparing both options:

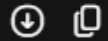
Option A: Using your original custom display methods (Good)

Inside your loop in `ContractList` , you have to explicitly call your custom method name:

Option A: Using your original custom display methods (Good)

Inside your loop in `ContractList`, you have to explicitly call your custom method name:

Java

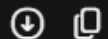


```
public void displayAllContracts() {  
    for (Contract c : listAllContracts) {  
        c.displayContractInfo(); // You must remember this exact name  
    }  
}
```

Option B: Using the `@Override toString()` method (Better & Cleaner)

If you override `toString()` inside `Contract.java`, look at how simple your `ContractList` code becomes:

Java



```
public void displayAllContracts() {  
    for (Contract c : listAllContracts) {  
        System.out.println(c); // Java automatically looks for the @Override toString  
    }  
}
```

Summary Recommendation for School

For your `ContractList` methods, leave `@Override` off because those are custom management tasks.

Summary Recommendation for School

For your `ContractList` methods, leave `@Override` off because those are custom management tasks.

But if you want to show your teacher you truly understand Java objects, override the `toString()` method inside `Contract.java` and `Salary.java`. Teachers love seeing this in Semester 2 because it proves you know that every class inherits from the master `Object` class, and it keeps your loops looking clean and easy to read!